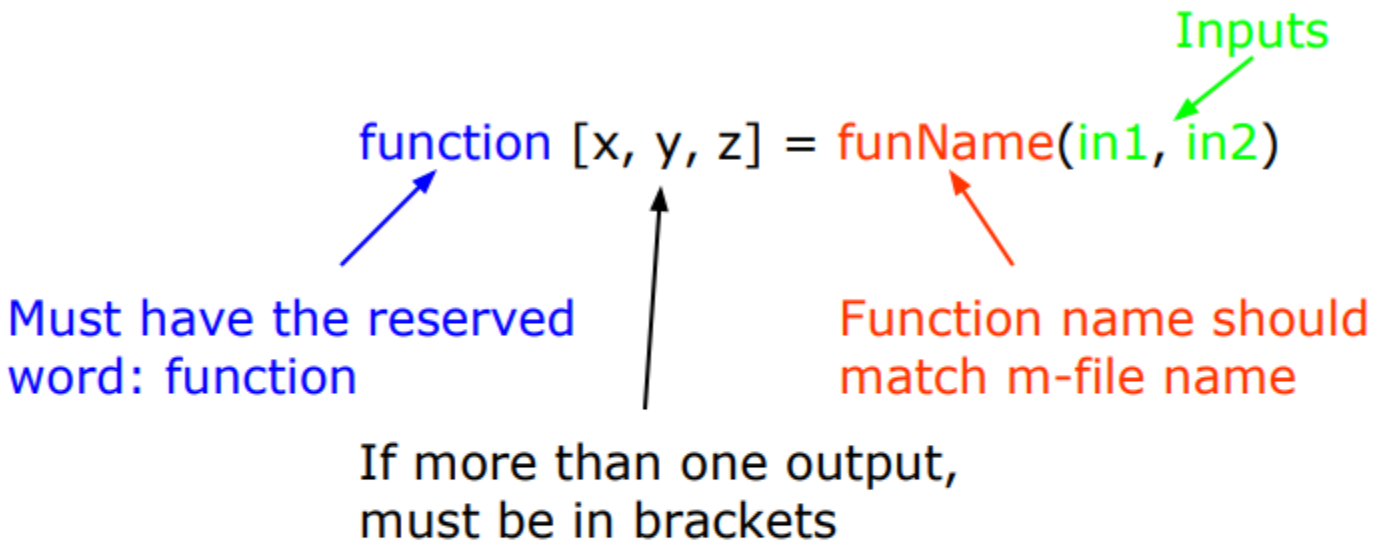
Matlab

- Some comments about the function declaration



- No need for return: MATLAB 'returns' the variables whose names match those in the function declaration (though, you can use **`return`** to break and go back to invoking function)
- Variable scope: Any variable created within the function but not returned disappears after the function stops running (They're called "local variables")

- We're familiar with
  - » `zeros`
  - » `size`
  - » `length`
  - » `sum`

- Look at the help file for size by typing
  - » `help size`

- The help file describes several ways to invoke the function
  - ➢ D = SIZE(X)
  - ➢ [M,N] = SIZE(X)
  - ➢ [M1,M2,M3,...,MN] = SIZE(X)
  - ➢ M = SIZE(X,DIM)

- MATLAB functions are generally overloaded
  - ➢ Can take a variable number of inputs
  - ➢ Can return a variable number of outputs

- What would the following commands return:

```
» a=zeros(2,4,8); %n-dimensional matrices are OK
» D=size(a)
» [m,n]=size(a)
» [x,y,z]=size(a)
» m2=size(a,2)
```

- You can overload your own functions by having variable number of input and output arguments (see **varargin**, **nargin**, **varargout**, **nargout**)

Eigen Value & Eigen Vector in MATLAB

```
[V, D] = eig(A);
```

A: Input square matrix.

V: A matrix whose columns are the eigenvectors of A.

D: A diagonal matrix containing the eigenvalues of A.

Basic Eigenvalue and Eigenvector Calculation

```
% Define a matrix
A = [4, -2; 1, 1];

% Compute eigenvalues and eigenvectors
[V, D] = eig(A);

% Display results
disp('Eigenvalues (D):');
disp(D);
disp('Eigenvectors (V):');
disp(V);
```

D: Diagonal matrix with eigenvalues

D =

3	0
0	2

V: Columns are the eigenvectors

V =

2	-1
1	1

This means:
Eigenvalue λ_1=3, eigenvector [2;1][2;1].

Eigenvalue λ_2 =2, eigenvector [−1;1][−1;1].

Verifying Eigenvalues and Eigenvectors

An eigenvalue-eigenvector relationship satisfies $A \cdot v = \lambda \cdot v$. Let's verify this:

```
% Verify for first eigenvalue and eigenvector
lambda1 = D(1, 1);
v1 = V(:, 1);

% Check A * v1 == lambda1 * v1
disp('Check A * v1:');
disp(A * v1);

disp('Check lambda1 * v1:');
disp(lambda1 * v1);
```

Eigenvalues Only

If you only need eigenvalues (not eigenvectors), use:

```
% Define a matrix
A = [3, -2; 1, 0];

% Compute eigenvalues
eigenvalues = eig(A);

% Display eigenvalues
disp('Eigenvalues:');
disp(eigenvalues);
```

Try to Find an Invertible Matrix PPP Such That B=P−1APB = P^{-1} A PB=P−1AP

```
A = [2 1; 0 2];
P = [1 1; 0 1];
B = inv(P) * A * P;

disp(B);  % Check if B equals the second matrix
```

If two matrices have the same:

- Characteristic polynomial
- Trace
- Determinant
- Eigenvalues with the same multiplicities

```
A = [2 1; 0 2];
B = [2 0; 0 2];

eigA = eig(A);
eigB = eig(B);

traceA = trace(A);
traceB = trace(B);

detA = det(A);
detB = det(B);

similar = isequal(sort(eigA), sort(eigB)) && traceA == trac
&& detA == detB;
disp(similar)
```

Diagonalization

Diagonalization expresses $A=V \cdot D \cdot V^{-1}$, where V contains eigenvectors and D is the diagonal matrix of eigenvalues.

```
% Define a matrix
A = [5, 4; 2, 3];

% Compute eigenvalues and eigenvectors
[V, D] = eig(A);

% Reconstruct A using V, D
A_reconstructed = V * D * inv(V);

% Display results
disp('Original Matrix A:');
disp(A);

disp('Reconstructed Matrix A:');
disp(A_reconstructed);
```

Complex Eigenvalues

If A has complex eigenvalues, MATLAB will handle them automatically.

```
% Define a matrix with complex eigenvalues
A = [0, -1; 1, 0];

% Compute eigenvalues and eigenvectors
[V, D] = eig(A);

% Display results
disp('Eigenvalues (D):');
disp(D);
disp('Eigenvectors (V):');
disp(V);
```

Output:
D: Diagonal matrix with complex eigenvalues

```
D =
   0 + 1i   0
   0        0 - 1i
```

Some Matrix Functions in MATLAB

X = ones(r,c)	Creates matrix full of ones
X = zeros(r,c)	Creates matrix full of zeros
A = diag(x)	Creates squared matrix with vector x in diagonal
[r,c] = size(A)	Return dimensions of matrix A
X = A’	Transposed matrix
X = inv(A)	Inverse matrix squared matrix
X = pinv(A)	Pseudo inverse
d = det(A)	Determinant
[X,D] = eig(A)	Eigenvalues and eigenvectors
[U,D,V] = svd(A)	singular value decomposition

Some Frequently Used Functions

```
>> magic(n)     % creates a special n x n  matrix; handy for testing
>> zeros(n,m)   % creates n x m matrix of zeroes (0)
>> ones(n,m)    % creates n x m matrix of ones (1)
>> rand(n,m)    % creates n x m matrix of random numbers
>> repmat(a,n,m)    % replicates a by n rows and m columns
>> diag(M)       % extracts the diagonals of a matrix M
>> help elmat   % list all elementary matrix operations ( or elfun)
>> abs(x);       % absolute value of x
>> exp(x);       % e to the x-th power
>> fix(x);        % rounds x to integer towards 0
>> log10(x);     % common logarithm of x to the base 10
>> rem(x,y);     % remainder of x/y
>> mod(x, y);   % modulus after division – unsigned rem
>> sqrt(x);       % square root of x
>> sin(x);        % sine of x; x in radians
>> acoth(x)     % inversion hyperbolic cotangent of x
```

Some Frequently Used Functions

```
>> magic(n)     % creates a special n x n  matrix; handy for testing
>> zeros(n,m)   % creates n x m matrix of zeroes (0)
>> ones(n,m)    % creates n x m matrix of ones (1)
>> rand(n,m)    % creates n x m matrix of random numbers
>> repmat(a,n,m)    % replicates a by n rows and m columns
>> diag(M)       % extracts the diagonals of a matrix M
>> help elmat   % list all elementary matrix operations ( or elfun)
>> abs(x);       % absolute value of x
>> exp(x);       % e to the x-th power
>> fix(x);        % rounds x to integer towards 0
>> log10(x);     % common logarithm of x to the base 10
>> rem(x,y);     % remainder of x/y
>> mod(x, y);   % modulus after division – unsigned rem
>> sqrt(x);       % square root of x
>> sin(x);        % sine of x; x in radians
>> acoth(x)     % inversion hyperbolic cotangent of x
```

MATLAB Graphics

- Line plot
- Bar graph
- Surface plot
- Contour plot
- MATLAB tutorial on 2D, 3D visualization tools as well as other graphics packages available in our tutorial series.

MATLAB Graphics

plot(X,Y)
plot(X,Y,LineSpec)
plot(X1,Y1,...,Xn,Yn)
plot(X1,Y1,LineSpec1,...,Xn,Yn,LineSpecn)
plot(Y)
plot(Y,LineSpec)
plot(tbl,xvar,yvar)
plot(tbl,yvar)
plot(ax,___)
plot(___,Name,Value)
p = plot(___)

```
x = linspace(-2*pi,2*pi);
y1 = sin(x);
y2 = cos(x);
plot(x,y1,x,y2);
y3 = sin(x-0.5);
plot(x,y1, 'g' ,x,y2, 'b--o' ,x,y3, 'c*' )
```

MATLAB Graphics

line style	explanation	result line
"-"	solid line	————
"--"	dashed line	- – – – –
":"	dotted line	.....
"-."	dashed line	–·–·–·–·

marker	explanation	Resulting Marker
"o"	one	○
"+"	plus	+
"*"	asterisk	✳
"."	dot	•
"x"	cross	×
"_"	horizontal line	—
"|"	vertical line	|
"square"	square	□
"diamond"	Diamond	◇
"^"	upward pointing triangle	△
"v"	downward pointing triangle	▽
">"	right-pointing triangle	▷
"<"	left-pointing triangle	◁
"pentagram"	pentagram	☆
"hexagram"	hexagram	✡

MATLAB Graphics

color name	short name	RGB 3 colors	shape
"red"	"r"	[1 0 0]	
"green"	"g"	[0 1 0]	
"blue"	"b"	[0 0 1]	
"cyan"	"c"	[0 1 1]	
"magenta"	"m"	[1 0 1]	
"yellow"	"y"	[1 1 0]	
"black"	"k"	[0 0 0]	
"white"	"w"	[1 1 1]	

MATLAB Graphics

```
p1 = [2 3]; % First Point
p2 = [9 8]; % Second Point
dp = p2-p1; % Difference
figure
quiver(p1(1),p1(2),dp(1),dp(2),0)
grid
axis([0 10 0 10])
text(p1(1),p1(2), sprintf('(%.0f,%.0f)',p1))
text(p2(1),p2(2), sprintf('(%.0f,%.0f)',p2))
```

Contour

contour(Z)
contour(X,Y,Z)
contour(___,levels)
contour(___,LineSpec)
contour(___,Name,Value)
contour(ax,___)
M = contour(___)
[M,c] = contour(___)

contour(Z) Z Creates a contour plot containing the isolines of the matrix , Z where contains the height values in the x - y plane. MATLAB ® automatically selects the contour lines to display. Z The column and row indices of are the x and y coordinates in the plane, respectively.

contour(X,Y,Z)Specifies the x and y, Z coordinates of the values contained in .

contour(___,levels) Specifies the contour lines to display in the syntax listed above as the last argument. If specified as levelsa scalar value, contour lines are displayed at automatically selected levels (heights). To draw contour lines at specific heights, specify as a vector of monotonically increasing values. To draw contour lines at a single height ( ), specify as a two-element row vector .nnlevelsklevels[k k]

contour(___,LineSpec) Specifies the style and color of the contour lines.

contour(___,Name,Value) Specifies additional options for the contour plot using one or more name-value pair arguments. Specify options after all other input arguments. For a list of properties, see Contour Properties .

contour(ax,___) Displays a contour plot on the target axes. In the syntax listed above, specify the axes as the first argument.

M = contour(___) Returns a contour matrix containing the (x , y) coordinates of the vertices at each level .M

[M,c] = contour(___) Returns a contour matrix and a contour object . Use to set properties cafter displaying the contour plot .c

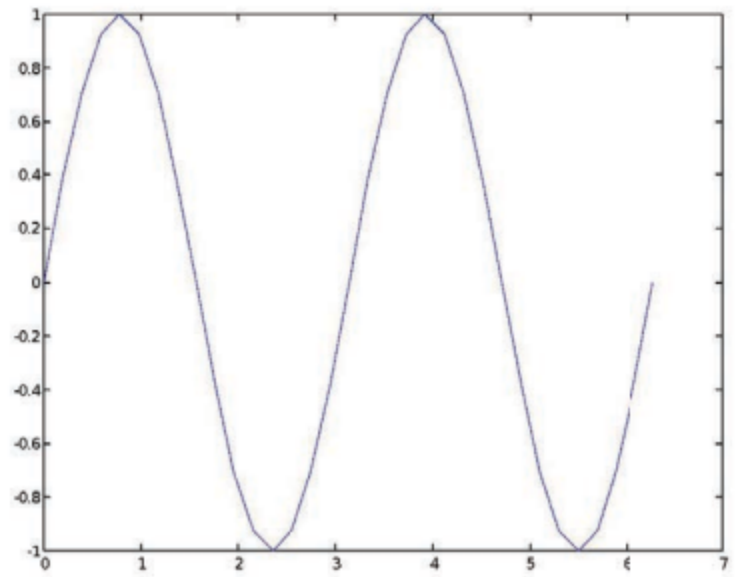
```
x = linspace(-2*pi,2*pi);
y = linspace(0,4*pi);
[X,Y] = meshgrid(x,y);
Z = sin(X)+cos(Y);
contour(X,Y,Z)
```

[X,Y] = meshgrid(x,y). Returns a two-dimensional grid coordinate based on the coordinates contained in vector x and . is a matrix where each row is a copy of and is a matrix where each column is a copy of . The grid represented by coordinates and has rows and columns

- Write a function with the following declaration:
  `function plotSin(f1)`

- In the function, plot a sine wave with frequency f1, on the interval [0,2π]: $\sin(f_1 x)$

- To get good sampling, use 16 points per period.



```
function plotSin(f1)
% plotSin - Plots a sine wave sin(f1 * x) over [0, 2*pi]
% using 16 points per period.


points_per_period = 16;
total_points = points_per_period * f1;


x = linspace(0, 2*pi, total_points);
y = sin(f1 * x);


plot(x, y);
title(['Sine wave with frequency f1 = ', num2str(f1)])
xlabel('x');
ylabel('sin(f1 * x)');
grid on;
end
```

```
function plotsin(Fi)
  % plotsin: This function plots a sine wave with a specified frequency.
  %
  % Input:
  %   Fi - Frequency of the sine wave in Hz
  %      Default value is 1 Hz if no input is provided.

  if nargin < 1
    Fi = 1; % Default frequency
    disp('Frequency not provided. Using default frequency of 1 Hz.');
  end

  % Validate the frequency
  if Fi <= 0
    error('Frequency Fi must be a positive number.');
  end

  % Define the time vector
  Fs = 1000; % Sampling frequency in Hz
  T = 1/Fs;  % Sampling period
  t = 0:T:1; % Time vector from 0 to 1 second

  % Generate the sine wave
  y = sin(2 * pi * Fi * t);

  % Plot the sine wave
  figure;
  plot(t, y, 'b', 'LineWidth', 1.5);
  grid on;
  xlabel('Time (s)');
  ylabel('Amplitude');
  title(['Sine Wave with Frequency ', num2str(Fi), ' Hz']);
  xlim([0 max(t)]);
end
```